

TECHNICAL SPECIFICATION · CONFIDENTIAL

Saudi Government RAG Agent

From web scraping to user interaction

Monshaat · MISA · Visit Saudi · Graphify · Claude

What is this?

A government knowledge agent that answers questions from three Saudi sources.

Who is it for?

Any developer or stakeholder who needs to understand how the system is built.

What does it cover?

Full pipeline: scraping, extraction, graph build, agent, and user interface.

SECTION 1 — THE PROBLEM

Why This Agent Exists

Saudi Arabia has three major government digital platforms — Monshaat (SME support), MISA (foreign investment), and Visit Saudi (tourism). Each platform contains hundreds of pages of programs, regulations, eligibility criteria, and procedural guides. A foreign investor, an entrepreneur, or a tourist must navigate all of them manually, in a language they may not fully understand, across separate websites with no connection between them.

This agent solves that problem. It reads all three sources, builds a connected knowledge graph, and answers any question a user has — from a single conversational interface, in their language, with the source cited for every fact.

Without the agent

- Three separate websites to navigate
- No connection between programs
- No guidance on eligibility
- Manual search across hundreds of pages
- No cross-source awareness

With the agent

- One conversational interface
- Graph connects related programs
- Agent matches user to eligibility
- Instant retrieval from knowledge graph
- Cross-source traversal built-in

SECTION 2 — HOW IT WORKS

The Big Picture

The system has five logical layers. Each layer feeds the next. Understanding each layer is enough to understand the whole system — no deep technical knowledge required.

Layer 01 — Scraping

Read every page from Monshaat, MISA, and Visit Saudi

Layer 02 — Extraction

Convert all PDFs into readable text (markdown format)

Layer 03 — Graph Build

Graphify reads all files and maps concepts + links

Layer 04 — Agent Logic

Agent reads the graph and answers user questions

Layer 05 — User Interface

User picks their intent, asks freely, gets cited answers

Each layer is independent. If the government website updates its content, only Layer 01 and 03 need to re-run. The agent and interface stay unchanged.

SECTION 3 — DATA SOURCES

What We Scrape and Why

Three sources form the knowledge base. Each maps to one or more user personas. The sources are independent but linked inside the knowledge graph — so a question that spans two sources gets a complete, connected answer.

Monshaat → SME programs → Kafalah loan guarantees → Accelerators & incubators → SME law & definitions → Training programs Personas: Entrepreneur, Investor	MISA → Foreign investor licensing → RHQ program → Economic zones → FDI incentives → Startup registration Personas: Investor, Entrepreneur	Visit Saudi → Tourist e-visa → Destination guides → Hotel & booking paths → Cultural experiences → Events calendar Persona: Tourist
--	---	---

SECTION 4 — THE KNOWLEDGE GRAPH

What Is a Knowledge Graph?

A knowledge graph is a database of concepts and the relationships between them. Unlike a traditional database of rows and columns, a graph stores things as nodes (concepts) and edges (connections).

In this system, every government program, regulation, visa type, and eligibility rule becomes a node. The links between them — 'Kafalah requires MISA registration', 'Tourist visa precedes investment visa' — become edges. The agent traverses these edges to answer complex multi-part questions.

Why a graph instead of a simple search?

Simple search: finds pages that mention your keyword.

Graph search: finds connected concepts and follows their relationships — answering questions that span multiple programs, sources, and eligibility conditions.

- 71x fewer tokens per query vs reading raw files (Graphify benchmark)
- Grows sublinearly — new content enriches existing nodes
- Cross-source links answered in one traversal

How namespace works

Every node in the graph carries a namespace tag — its source. A node from Monshaat is tagged `monshaat::`, a MISA node is `misa::`, and so on. This allows the agent to start from the right cluster for each user persona while still following cross-source links when the answer requires it.

```
// Example node in the graph
{
  "id": "monshaat::kafalah_program",
  "namespace": "monshaat",
  "label": "Kafalah Guarantee Program",
  "content": "Kafalah provides loan guarantees...",
  "source_url": "https://monshaat.gov.sa/kafalah",
  "personas": ["entrepreneur", "investor"],
  "edges": [
    { "target": "monshaat::sme_eligibility", "type": "REQUIRES" },
    { "target": "misa::investor_license", "type": "SUPPORTS" }
  ]
}
```

Phase-by-Phase Build Plan

The system is built in eight sequential phases. Each phase produces a concrete output that feeds the next. A developer can implement and test each phase independently.

1

Web Scraping

Playwright + httpx · per source folder

A single scraper function runs against each source URL. It uses Playwright (a headless browser) to handle JavaScript-rendered pages that a simple HTTP request would miss. For each page it visits, it:

- Converts the HTML to clean markdown using markdownify
- Injects a frontmatter header with source name, URL, and persona tags
- Saves the file to /knowledge_base/{source_name}/slug.md
- Detects any PDF links and hands them to the PDF extractor
- Follows internal links to crawl the full site

```
# Folder structure produced by the scraper
/knowledge_base
  /monshaat/
    kafalah_program.md      ← frontmatter: source=monshaat
    sme_definition.md
    accelerator_directory.md
  /misa/
    investor_license.md     ← frontmatter: source=misa
    rhq_program.md
  /visitsaudi/
    tourist_evisa.md        ← frontmatter: source=visitsaudi
    alula_destination.md
  /cross_entity/
    cross_entity_edges.json ← manually curated links
```

2

PDF Extraction

pdfplumber → GLM-OCR fallback

Every PDF found during scraping goes through a two-step extraction pipeline. The output is always clean markdown with the same frontmatter structure as scraped pages — so Graphify treats PDFs identically to web pages.

Step 1 — pdfplumber

Fast, free, deterministic.
Works for digital PDFs where text is selectable.
Covers ~80-90% of cases.
No API call needed.

Step 2 — GLM-OCR (fallback)

Runs only if pdfplumber returns empty text.
Handles scanned PDFs.
0.9B model — self-hostable.
Outputs native markdown.

3

Graphify Extraction

One run per source folder → merge

Graphify reads every markdown file in a source folder and extracts concepts and relationships using an LLM. It produces a graph.json file per source. A merge script then combines all three into one federated graph, injecting the namespace into every node ID.

```
# Run Graphify per source (terminal commands)
graphify extract ./knowledge_base/monshaat --output ./graphs/monshaat.json
graphify extract ./knowledge_base/misa --output ./graphs/misa.json
graphify extract ./knowledge_base/visitsaudi --output ./graphs/visitsaudi.json

# Then merge into one federated graph
python graph_builder.py
# → produces ./graphs/federated_graph.json
# with namespace injected into every node ID:
# monshaat::kafalah_program
# misa::investor_license
# visitsaudi::tourist_evisa
```

Cross-entity edges — links between nodes from different sources — are maintained in a manually curated JSON file. These capture relationships that Graphify misses because they span source documents (e.g. 'MISA license enables Kafalah application').

4

Graph Query Engine

In-memory traversal · namespace filtering

The federated graph is loaded into memory once at startup. A lightweight Python class provides three operations used by the agent:

- `get_nodes_by_namespace` — returns all nodes for a given source
- `semantic_search` — finds top-k nodes matching a query within a namespace filter
- `traverse` — BFS walk from a set of start nodes up to `max_hops` depth

The semantic search in this specification uses keyword matching for simplicity. In production, replace it with vector embeddings (pgvector or Pinecone) for significantly better retrieval quality.

5

Agent Core

Claude API · persona routing · history management

The agent combines graph retrieval with LLM generation. For every user question it runs seven steps:

1. Persona → namespaces

Map the selected persona to its primary source namespaces

2. Semantic search

Find top-5 nodes matching the question within those namespaces

3. Follow-up traversal

If prior active nodes exist, traverse 2 hops from them too

4. Format context

Combine nodes into a context block with source attribution

5. LLM call

Send context + history + question to Claude

6. Update session

Store active node IDs for next turn's traversal

7. Update history

Append turn to history, cap at 6 turns (12 messages)

6

API Layer

FastAPI · three endpoints

A FastAPI backend exposes three endpoints. Session state is held in memory (use Redis for production scale).

POST /session/start

User selects persona.
Session created.
Greeting returned.
Entry namespace set.

POST /session/message

User sends question.
Agent queries graph.
LLM generates answer.
History updated.

POST /session/switch

User changes intent.
Persona updated.
Active nodes reset.
Conversation continues.

7

User Interaction

Persona selection · Q&A · topic switching

The user opens the interface and sees three choices. One tap sets the full session context. Every subsequent question inherits that context automatically. The user can switch intent at any time.

```
UI: 'What brings you to Saudi Arabia today?'
    [ Investor ] [ Entrepreneur ] [ Tourist ]

User: clicks Entrepreneur

UI: 'Welcome. I can help you find SME programs, Kafalah
    financing, and startup support. What would you like to know?'

User: 'What financing is available for a new startup?'

Agent: Graph traversal from monshaat:: namespace →
    monshaat::kafalah_program
    monshaat::sme_definition
    (cross-entity edge) → misa::startup_license

    Answer: 'Monshaat offers the Kafalah loan guarantee
    program... [source: monshaat.gov.sa/kafalah]
    To access this, you first need MISA registration...
    [source: misa.gov.sa/startup]'
```

```
User: 'What are the eligibility criteria?'

Agent: Continues traversal from active nodes →
    monshaat::kafalah_eligibility
    Answer: 'To be eligible for Kafalah you must...'
```

8

Maintenance

Weekly cron · add new sources in 4 steps

Government content changes slowly but it does change. A weekly cron job re-scrapes all sources and rebuilds the graph. Adding a new source requires only four configuration changes — no architectural modifications needed.

```
# Weekly cron job (runs every Monday at 2am)
0 2 * * 1 python scraper.py \
    && python pdf_extractor.py \
    && graphify extract ./knowledge_base/monshaat --output ./graphs/monshaat.json \
    && graphify extract ./knowledge_base/misa --output ./graphs/misa.json \
    && graphify extract ./knowledge_base/visitsaudi --output ./graphs/visitsaudi.json \
    && python graph_builder.py

# To add a new source (e.g. Saudi Tourism Authority):
# 1. Add URL to SOURCES dict in scraper.py
# 2. Create /knowledge_base/sta/ folder
# 3. Add 'sta' to PERSONA_NAMESPACE_MAP in config.py
# 4. Re-run scraper + graphify extract + graph_builder
```


Tools and Libraries

Layer	Tool	Purpose
Scraping	Playwright	JS-rendered page crawling
Scraping	httpx	Fast static page fetching
Scraping	markdownify	HTML → markdown conversion
PDF extraction	pdfplumber	Digital PDF text extraction
PDF extraction	GLM-OCR	Scanned PDF OCR fallback
Graph build	Graphify	Concept + relationship extraction
Graph build	graph_builder.py	Namespace injection + merge
Agent	Claude API	LLM generation
Agent	LangChain	Conversation + chain management
API	FastAPI	REST backend
Session	In-memory dict	Session state (Redis for prod)

SECTION 7 — KEY DESIGN DECISIONS

Why These Choices Were Made

One graph, not three

Splitting graphs per source breaks cross-source answers. A federated graph with namespace tags gives the same isolation without losing

Upfront persona selection

Dynamic classification adds one LLM call per question and can misclassify overlapping queries. Explicit upfront selection is 100% accurate

Persona = entry point, not a filter

Locking a persona to only its source namespaces breaks answers that span sources. Persona sets where traversal starts — the graph still

History capped at 6 turns

Full conversation history grows token cost linearly. Government Q&A rarely requires context older than 3-4 turns. Capping at 6 turns (12 m

Markdown as the universal format

Both scraped pages and extracted PDFs output markdown. Graphify reads markdown natively. This common format eliminates format-spe

End-to-End Pipeline at a Glance

01. Playwright scrapes all three government websites
02. pdfplumber + GLM-OCR converts every PDF to markdown
03. Frontmatter injected into every file (source, URL, personas)
04. Files saved to /knowledge_base/{source}/ folders
05. Graphify runs per folder → extracts nodes + edges
06. Merge script combines graphs → one federated_graph.json
07. Namespace injected into every node ID (source::node_id)
08. Cross-entity edges merged from manual curation file
09. FastAPI backend exposes three endpoints
10. User opens interface → selects persona (one tap)
11. User asks question → agent queries graph from entry namespace
12. Agent traverses edges → retrieves relevant nodes across sources
13. Claude generates answer with source attribution
14. History capped at 6 turns → active nodes carried forward
15. Weekly cron rebuilds graph from fresh scrape

Ready to build?

Start with Phase 1 (scraper.py) and validate markdown quality before running Graphify.